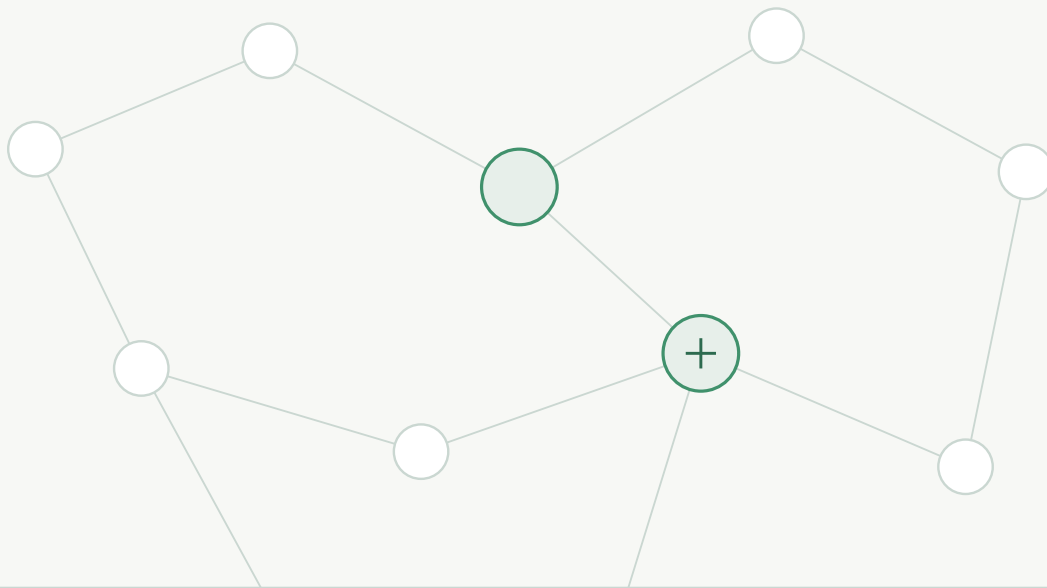


Laboratorio de Red Empresarial en GNS3

Segmentación por VLANs, ruteo inter-VLAN, ACLs, NAT/PAT y hosts Docker con sockets TCP en C++.



AUTOR

Fernando Segundo

Estudiante de Tecnólogo en Informática - UTEC

VLANs / VTP / Trunks

Router-on-a-Stick / ACLs

NAT-PAT / Docker / C++

Contenido

- 1. Introducción 3
- 2. Topología utilizada 3
- 3. Direccionamiento IP 4
- 4. DHCP 5
- 5. VLANs y VTP 6
 - 5.1 Base de datos de VLANs en los switches 6
 - 5.2 Estado de VTP en los switches 7
 - 5.3 Verificación de enlaces trunk 802.1Q..... 8
- 6. Router-on-a-Stick 9
- 7. ACLs de segmentación..... 10
 - 7.1 ACL de Administración 10
 - 7.2 ACL de Ventas..... 10
 - 7.3 ACL de Aplicación 10
- 8. Hosts de aplicación en Docker 12
 - 8.1 Servidor GNS3 y acceso mediante Tailscale 12
 - 8.2 Imagen Docker utilizada 13
- 9. Aplicación en C++ con sockets TCP..... 14
 - 9.1 Código del servidor..... 14
 - 9.2 Código del cliente 15
- 10. Salida a Internet mediante NAT/PAT 16
- 11. DNS 17
 - 11.1 Diferencia entre conectividad por IP pública y resolución DNS..... 17
- 12. Problemas encontrados durante la implementación..... 18
 - 12.1 DHCP bloqueado por ACL 18
 - 12.2 Interfaz trunk incorrecta 18
 - 12.3 Ruta por defecto hacia Internet 18
 - 12.4 Persistencia en Docker 18

1. Introducción

La red se armó en GNS3 tomando como base una empresa con tres sectores: Administración, Ventas y Aplicación. Cada sector quedó separado en una VLAN distinta. Administración y Ventas reciben dirección IP mediante DHCP, mientras que los equipos de aplicación usan direccionamiento manual.

Para comunicar las VLANs se usó Router-on-a-Stick sobre un router Cisco 7200. La creación y propagación de VLANs se realizó con VTP en los switches ESW basados en c3725. El acceso entre redes se limitó mediante ACLs en el router.

Para ServerApp y ClientApp se utilizaron contenedores Docker, autorizados por el docente para este trabajo. Ambos equipos se conectaron a la VLAN de aplicación y se usaron para validar la comunicación mediante una aplicación cliente-servidor en C++ con sockets TCP por el puerto 8080.

2. Topología utilizada

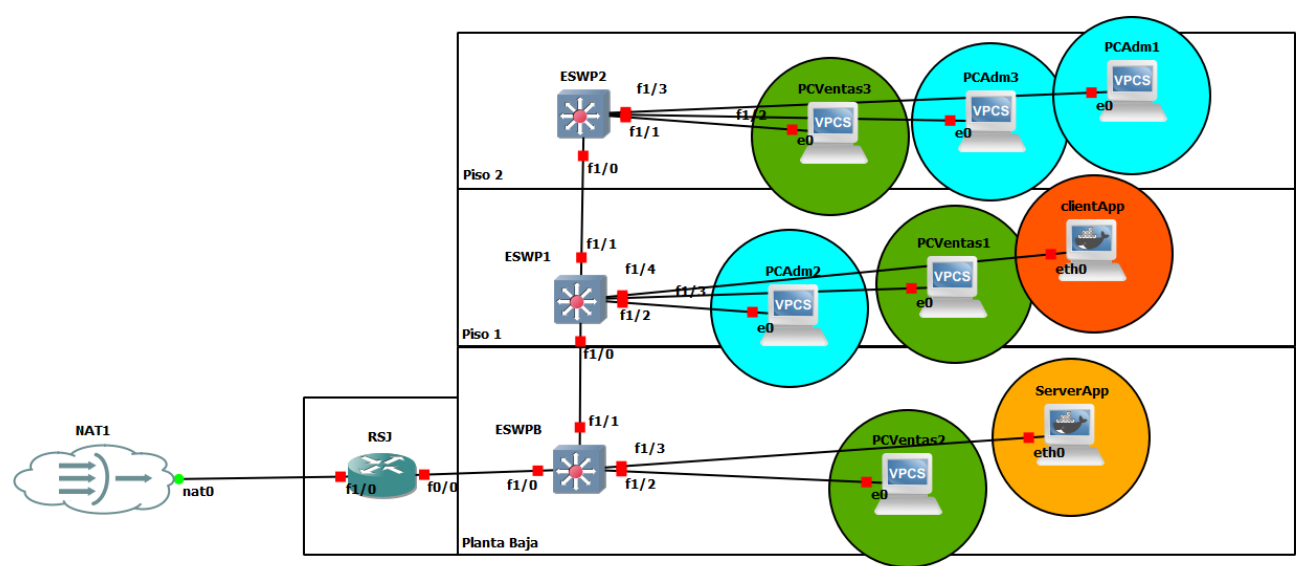


Figura 1. Topología general del laboratorio en GNS3.

La topología tiene tres niveles físicos representados como Planta Baja, Piso 1 y Piso 2. Los switches se enlazan mediante troncales y el switch de Planta Baja conecta hacia el router RSJ. El router también se conecta al nodo NAT de GNS3 para dar salida a Internet a los equipos necesarios.

Equipo	Función
RSJ	Router Cisco 7200. Router-on-a-Stick, DHCP, ACLs y NAT/PAT.
ESWPB, ESWP1, ESWP2	Switches ESW sobre c3725. VLANs, VTP y enlaces trunk.
PCAdm	Hosts de Administración. VLAN 10. DHCP.
PCVentas	Hosts de Ventas. VLAN 20. DHCP.
ServerApp	Contenedor Docker en VLAN 30. IP manual 192.168.1.130.

ClientApp	Contenedor Docker en VLAN 30. IP manual 192.168.1.190.
NAT1	Nodo NAT de GNS3 para salida a Internet.

3. Direccionamiento IP

La red original indicada fue 192.168.1.0/24. Se subdividió en cuatro subredes tomando 2 bits del campo de host, por lo que la nueva máscara quedó en /26, equivalente a 255.255.255.192.

Máscara /24: 11111111.11111111.11111111.00000000
Máscara /26: 11111111.11111111.11111111.11000000
Bits tomados: 2
Subredes generadas: $2^2 = 4$
Direcciones por subred: $2^6 = 64$
Hosts utilizables por subred: $64 - 2 = 62$

La primera IP válida de cada subred se usó como puerta de enlace. En la red de aplicación, ServerApp usa la segunda IP válida y ClientApp usa la última IP válida, como pide la letra.

Uso	VLAN	Red	Máscara	Gateway	Rango utilizable	Broadcast
Administración	10	192.168.1.0/26	255.255.255.192	192.168.1.1	192.168.1.1 - 192.168.1.62	192.168.1.63
Ventas	20	192.168.1.64/26	255.255.255.192	192.168.1.65	192.168.1.65 - 192.168.1.126	192.168.1.127
Aplicación	30	192.168.1.128/26	255.255.255.192	192.168.1.129	192.168.1.129 - 192.168.1.190	192.168.1.191
Reservada	40	192.168.1.192/26	255.255.255.192	192.168.1.193	192.168.1.193 - 192.168.1.254	192.168.1.255
Host	IP		Máscara	Gateway	Observación	
ServerApp	192.168.1.130		255.255.255.192	192.168.1.129	Segunda IP válida de la subred APP.	
ClientApp	192.168.1.190		255.255.255.192	192.168.1.129	Última IP válida de la subred APP.	

Figura 2. Estado de interfaces del router RSJ mediante show ip interface brief.

```
RSJ#show ip int b
```

Interface	IP-Address	OK?	Method	Status	Protocol
FastEthernet0/0	unassigned	YES	NVRAM	up	up
FastEthernet0/0.10	192.168.1.1	YES	NVRAM	up	up
FastEthernet0/0.20	192.168.1.65	YES	NVRAM	up	up
FastEthernet0/0.30	192.168.1.129	YES	NVRAM	up	up
FastEthernet1/0	192.168.122.156	YES	DHCP	up	up
FastEthernet1/1	unassigned	YES	NVRAM	administratively down	down
GigabitEthernet2/0	unassigned	YES	NVRAM	administratively down	down
NVI0	unassigned	NO	unset	up	up

4. DHCP

El servicio DHCP se configuró en el router RSJ solamente para las VLAN Administración y Ventas. La VLAN de aplicación quedó con direccionamiento manual para ServerApp y ClientApp.

```
no ip dhcp use vrf connected
ip dhcp excluded-address 192.168.1.1
ip dhcp excluded-address 192.168.1.65
ip dhcp excluded-address 192.168.1.129
ip dhcp pool ADM
  network 192.168.1.0 255.255.255.192
  default-router 192.168.1.1
  dns-server 8.8.8.8
ip dhcp pool VENTAS
  network 192.168.1.64 255.255.255.192
  default-router 192.168.1.65
  dns-server 8.8.8.8
```

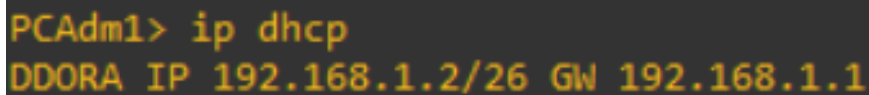
Se excluyeron las direcciones usadas como puertas de enlace para evitar que sean entregadas a clientes. Los pools ADM y VENTAS entregan IP, máscara, gateway y DNS a sus hosts.

Figura 3. Configuración DHCP del router RSJ.



```
RSJ#show run | section dhcp
no ip dhcp use vrf connected
ip dhcp excluded-address 192.168.1.1
ip dhcp excluded-address 192.168.1.65
ip dhcp excluded-address 192.168.1.129
ip dhcp pool ADM
  network 192.168.1.0 255.255.255.192
  default-router 192.168.1.1
  dns-server 8.8.8.8
ip dhcp pool VENTAS
  network 192.168.1.64 255.255.255.192
  default-router 192.168.1.65
  dns-server 8.8.8.8
ip address dhcp
```

Figura 4. PCAdm1 obteniendo dirección IP por DHCP.



```
PCAdm1> ip dhcp
DDORA IP 192.168.1.2/26 GW 192.168.1.1
```

Figura 5. PCVentas1 obteniendo dirección IP por DHCP.

```
PCVentas1> ip dhcp
DDORA IP 192.168.1.66/26 GW 192.168.1.65
```

5. VLANs y VTP

Las VLANs se crearon en los switches y se propagaron usando VTP. El dominio configurado fue REDES y se trabajó con VTP versión 2.

VLAN	Nombre	Uso
10	ADM	Administración
20	VENTAS	Ventas
30	APP	Aplicación

```
ESWPB#vlan database
```

```
ESWPB(vlan)#vlan 10 name ADM
```

```
ESWPB(vlan)#vlan 20 name VENTAS
```

```
ESWPB(vlan)#vlan 30 name APP
```

Los enlaces entre switches y el enlace hacia el router se configuraron como trunk 802.1Q para transportar las VLAN 10, 20 y 30.

5.1 Base de datos de VLANs en los switches

Figura 6. Base de datos de VLANs en ESWPB.

```
ESWPB#show vlan-s
```

VLAN	Name	Status	Ports
1	default	active	Fa1/4, Fa1/5, Fa1/6, Fa1/7 Fa1/8, Fa1/9, Fa1/10, Fa1/11 Fa1/12, Fa1/13, Fa1/14, Fa1/15
10	ADM	active	
20	VENTAS	active	Fa1/2
30	APP	active	Fa1/3
1002	fddi-default	active	
1003	trcrf-default	active	
1004	fddinet-default	active	
1005	trbrf-default	active	

Figura 7. Base de datos de VLANs en ESWP1.

```
ESWP1#show vlan-s
```

VLAN	Name	Status	Ports
1	default	active	Fa1/5, Fa1/6, Fa1/7, Fa1/8 Fa1/9, Fa1/10, Fa1/11, Fa1/12 Fa1/13, Fa1/14, Fa1/15
10	ADM	active	Fa1/2
20	VENTAS	active	Fa1/3
30	APP	active	Fa1/4
1002	fddi-default	active	
1003	trcrf-default	active	
1004	fddinet-default	active	
1005	trbrf-default	active	

Figura 8. Base de datos de VLANs en ESWP2.

```
ESWP2#show vlan-s
```

VLAN	Name	Status	Ports
1	default	active	Fa1/4, Fa1/5, Fa1/6, Fa1/7 Fa1/8, Fa1/9, Fa1/10, Fa1/11 Fa1/12, Fa1/13, Fa1/14, Fa1/15
10	ADM	active	Fa1/2, Fa1/3
20	VENTAS	active	Fa1/1
30	APP	active	
1002	fddi-default	active	
1003	trcrf-default	active	
1004	fddinet-default	active	
1005	trbrf-default	active	

5.2 Estado de VTP en los switches

Figura 9. Estado VTP en ESWPB como servidor del dominio REDES.

```
ESWPB#show vtp status
VTP Version                : 2
Configuration Revision      : 2
Maximum VLANs supported locally : 36
Number of existing VLANs    : 8
VTP Operating Mode          : Server
VTP Domain Name              : REDES
VTP Pruning Mode             : Disabled
VTP V2 Mode                  : Enabled
VTP Traps Generation        : Disabled
MD5 digest                  : 0xE0 0x91 0x03 0x89 0xFE 0xE6 0xF8 0x49
Configuration last modified by 0.0.0.0 at 3-1-02 00:26:28
Local updater ID is 0.0.0.0 (no valid interface found)
```


Figura 10. Estado VTP en ESWP1 como cliente del dominio REDES.

```
ESWP1#show vtp status
VTP Version                : 2
Configuration Revision      : 2
Maximum VLANs supported locally : 36
Number of existing VLANs    : 8
VTP Operating Mode          : Client
VTP Domain Name             : REDES
VTP Pruning Mode            : Disabled
VTP V2 Mode                 : Enabled
VTP Traps Generation        : Disabled
MD5 digest                  : 0xE0 0x91 0x03 0x89 0xFE 0xE6 0xF8 0x49
Configuration last modified by 0.0.0.0 at 3-1-02 00:26:28
```

Figura 11. Estado VTP en ESWP2 como cliente del dominio REDES.

```
ESWP2#show vtp status
VTP Version                : 2
Configuration Revision      : 2
Maximum VLANs supported locally : 36
Number of existing VLANs    : 8
VTP Operating Mode          : Client
VTP Domain Name             : REDES
VTP Pruning Mode            : Disabled
VTP V2 Mode                 : Enabled
VTP Traps Generation        : Disabled
MD5 digest                  : 0xE0 0x91 0x03 0x89 0xFE 0xE6 0xF8 0x49
Configuration last modified by 0.0.0.0 at 3-1-02 00:26:28
```

5.3 Verificación de enlaces trunk 802.1Q

Figura 12. Troncales activas en ESWPB.

```
ESWPB#show int tr

Port      Mode      Encapsulation  Status      Native vlan
Fa1/0     on        802.1q         trunking    1
Fa1/1     on        802.1q         trunking    1

Port      Vlans allowed on trunk
Fa1/0     1,10,20,30,1002-1005
Fa1/1     1,10,20,30,1002-1005

Port      Vlans allowed and active in management domain
Fa1/0     1,10,20,30
Fa1/1     1,10,20,30

Port      Vlans in spanning tree forwarding state and not pruned
Fa1/0     1,10,20,30
Fa1/1     1,10,20,30
```


Figura 13. Troncales activas en ESWP1.

```
ESWP1#show int trunk

Port      Mode      Encapsulation  Status      Native vlan
Fa1/0     on        802.1q         trunking    1
Fa1/1     on        802.1q         trunking    1

Port      Vlans allowed on trunk
Fa1/0     1,10,20,30,1002-1005
Fa1/1     1,10,20,30,1002-1005

Port      Vlans allowed and active in management domain
Fa1/0     1,10,20,30
Fa1/1     1,10,20,30

Port      Vlans in spanning tree forwarding state and not pruned
Fa1/0     1,10,20,30
Fa1/1     1,10,20,30
```

Figura 14. Troncal activa en ESWP2.

```
ESWP2#show int tr

Port      Mode      Encapsulation  Status      Native vlan
Fa1/0     on        802.1q         trunking    1

Port      Vlans allowed on trunk
Fa1/0     1,10,20,30,1002-1005

Port      Vlans allowed and active in management domain
Fa1/0     1,10,20,30

Port      Vlans in spanning tree forwarding state and not pruned
Fa1/0     1,10,20,30
```

6. Router-on-a-Stick

El router RSJ realiza el ruteo entre VLANs mediante subinterfaces sobre FastEthernet0/0. Cada subinterfaz usa encapsulación 802.1Q y tiene asignada la primera IP válida de su subred.

```
interface FastEthernet0/0.10
 encapsulation dot1q 10
 ip address 192.168.1.1 255.255.255.192
 ip access-group VLAN10_IN in
```

```
interface FastEthernet0/0.20
 encapsulation dot1Q 20
 ip address 192.168.1.65 255.255.255.192
 ip access-group VLAN20_IN in

interface FastEthernet0/0.30
 encapsulation dot1Q 30
 ip address 192.168.1.129 255.255.255.192
 ip access-group VLAN30_IN in
 ip nat inside
 ip virtual-reassembly
```

La subinterfaz de la VLAN 30 también fue marcada como NAT inside, ya que desde esa red salen ServerApp y ClientApp hacia Internet.

7. ACLs de segmentación

Las ACLs se aplicaron de entrada en las subinterfaces del router. El criterio usado fue bloquear comunicación entre VLANs, pero permitir la comunicación necesaria entre ClientApp y ServerApp y la salida a Internet de los equipos autorizados.

7.1 ACL de Administración

```
ip access-list extended VLAN10_IN
deny ip 192.168.1.0 0.0.0.63 192.168.1.64 0.0.0.63
deny ip 192.168.1.0 0.0.0.63 192.168.1.128 0.0.0.63
permit ip 192.168.1.0 0.0.0.63 any
permit udp any eq bootpc any eq bootps
permit udp any eq bootps any eq bootpc
```

7.2 ACL de Ventas

```
ip access-list extended VLAN20_IN
deny ip 192.168.1.64 0.0.0.63 192.168.1.0 0.0.0.63
deny ip 192.168.1.64 0.0.0.63 192.168.1.128 0.0.0.63
permit ip 192.168.1.64 0.0.0.63 any
permit udp any eq bootpc any eq bootps
permit udp any eq bootps any eq bootpc
```

7.3 ACL de Aplicación

```
ip access-list extended VLAN30_IN
permit tcp 192.168.1.128 0.0.0.63 host 192.168.1.130 eq 8080
permit tcp host 192.168.1.130 192.168.1.128 0.0.0.63 established
deny ip 192.168.1.128 0.0.0.63 192.168.1.0 0.0.0.63
deny ip 192.168.1.128 0.0.0.63 192.168.1.64 0.0.0.63
permit ip 192.168.1.128 0.0.0.63 any
permit udp any eq bootpc any eq bootps
permit udp any eq bootps any eq bootpc
```

Durante las pruebas se detectó que DHCP no funcionaba si la ACL no contemplaba los puertos UDP 67 y 68. Por eso se agregaron reglas para permitir tráfico bootps/bootpc.

Figura 15. ACLs configuradas en el router RSJ.

```
RSJ#show access-lists
Standard IP access list NAT_SERVERS
 10 permit 192.168.1.190 (1 match)
 20 permit 192.168.1.130 (2 matches)
Extended IP access list VLAN10_IN
 10 deny ip 192.168.1.0 0.0.0.63 192.168.1.64 0.0.0.63
 20 deny ip 192.168.1.0 0.0.0.63 192.168.1.128 0.0.0.63
 30 permit ip 192.168.1.0 0.0.0.63 any
 40 permit udp any eq bootpc any eq bootps (27 matches)
 50 permit udp any eq bootps any eq bootpc
Extended IP access list VLAN20_IN
 10 deny ip 192.168.1.64 0.0.0.63 192.168.1.0 0.0.0.63
 20 deny ip 192.168.1.64 0.0.0.63 192.168.1.128 0.0.0.63
 30 permit ip 192.168.1.64 0.0.0.63 any
 40 permit udp any eq bootpc any eq bootps (27 matches)
 50 permit udp any eq bootps any eq bootpc
Extended IP access list VLAN30_IN
 10 permit tcp 192.168.1.128 0.0.0.63 host 192.168.1.130 eq 8080
 20 permit tcp host 192.168.1.130 192.168.1.128 0.0.0.63 established
 30 deny ip 192.168.1.128 0.0.0.63 192.168.1.0 0.0.0.63
 40 deny ip 192.168.1.128 0.0.0.63 192.168.1.64 0.0.0.63
 50 permit ip 192.168.1.128 0.0.0.63 any (16 matches)
 60 permit udp any eq bootpc any eq bootps
 70 permit udp any eq bootps any eq bootpc
```

Figura 16. Ping exitoso desde ClientApp hacia ServerApp.

```
/app # ping 192.168.1.130
PING 192.168.1.130 (192.168.1.130): 56 data bytes
64 bytes from 192.168.1.130: seq=0 ttl=64 time=2.234 ms
64 bytes from 192.168.1.130: seq=1 ttl=64 time=0.603 ms
64 bytes from 192.168.1.130: seq=2 ttl=64 time=1.019 ms
64 bytes from 192.168.1.130: seq=3 ttl=64 time=0.852 ms
64 bytes from 192.168.1.130: seq=4 ttl=64 time=0.817 ms
64 bytes from 192.168.1.130: seq=5 ttl=64 time=1.744 ms
^C
--- 192.168.1.130 ping statistics ---
6 packets transmitted, 6 packets received, 0% packet loss
round-trip min/avg/max = 0.603/1.211/2.234 ms
```

Figura 17. Ping bloqueado desde Ventas hacia Administración.

```
PCVentas1> ping 192.168.1.2
*192.168.1.65 icmp_seq=1 ttl=255 time=39.870 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.65 icmp_seq=2 ttl=255 time=23.765 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.65 icmp_seq=3 ttl=255 time=23.291 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.65 icmp_seq=4 ttl=255 time=11.108 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.65 icmp_seq=5 ttl=255 time=22.417 ms (ICMP type:3, code:13, Communication administratively prohibited)
```

Figura 18. Ping bloqueado desde Administración hacia Ventas.

```
PCAdm1> ping 192.168.1.66
*192.168.1.1 icmp_seq=1 ttl=255 time=29.060 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.1 icmp_seq=2 ttl=255 time=14.252 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.1 icmp_seq=3 ttl=255 time=12.349 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.1 icmp_seq=4 ttl=255 time=24.961 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.1 icmp_seq=5 ttl=255 time=27.419 ms (ICMP type:3, code:13, Communication administratively prohibited)
```

Figura 19. Ping bloqueado desde Ventas hacia ServerApp.

```
PCVentas1> ping 192.168.1.130
*192.168.1.65 icmp_seq=1 ttl=255 time=34.352 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.65 icmp_seq=2 ttl=255 time=12.213 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.65 icmp_seq=3 ttl=255 time=14.607 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.65 icmp_seq=4 ttl=255 time=25.031 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.65 icmp_seq=5 ttl=255 time=7.935 ms (ICMP type:3, code:13, Communication administratively prohibited)
```

Figura 20. Ping bloqueado desde Administración hacia ClientApp.

```
PCAdm1> ping 192.168.1.190
*192.168.1.1 icmp_seq=1 ttl=255 time=29.482 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.1 icmp_seq=2 ttl=255 time=14.859 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.1 icmp_seq=3 ttl=255 time=24.012 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.1 icmp_seq=4 ttl=255 time=16.410 ms (ICMP type:3, code:13, Communication administratively prohibited)
*192.168.1.1 icmp_seq=5 ttl=255 time=13.967 ms (ICMP type:3, code:13, Communication administratively prohibited)
```

8. Hosts de aplicación en Docker

Aunque la letra menciona máquinas virtuales Ubuntu, para esta implementación se usaron contenedores Docker autorizados por el docente. Los contenedores cumplen el rol de ServerApp y ClientApp dentro de la VLAN 30.

8.1 Servidor GNS3 y acceso mediante Tailscale

El laboratorio se ejecutó sobre un servidor casero donde corre el servidor de GNS3. La administración del proyecto se realizó desde la GUI de GNS3 instalada en Windows, conectándose al servidor por medio de Tailscale. Esta VPN permite acceder al servidor de forma privada, sin exponer directamente el servicio de GNS3 a Internet, y facilita trabajar con los nodos del laboratorio desde otra máquina.

De esta forma, los routers, switches y contenedores se ejecutan en el servidor, mientras que la computadora Windows se usa principalmente como interfaz gráfica de administración.

8.2 Imagen Docker utilizada

Para ServerApp y ClientApp se utilizó una imagen Docker propia basada en Ubuntu 22.04. El Dockerfile se creó en el servidor casero y se usó como base para los contenedores de aplicación dentro de GNS3.

```
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt update && apt install -y \
    g++ \
    iproute2 \
    iputils-ping \
    net-tools \
    curl \
    telnet \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
```

Los paquetes incluidos en la imagen cumplen las siguientes funciones:

- ubuntu:22.04: imagen base del contenedor, elegida para trabajar con un entorno Linux conocido y compatible con las herramientas usadas.
- DEBIAN_FRONTEND=noninteractive: evita que la instalación de paquetes quede esperando confirmaciones durante la construcción de la imagen.
- g++: compilador utilizado para compilar las aplicaciones cliente y servidor desarrolladas en C++.
- iproute2: proporciona comandos como ip addr e ip route, necesarios para configurar direcciones IP y puerta de enlace dentro del contenedor.
- iputils-ping: permite realizar pruebas de conectividad con el comando ping.
- net-tools: agrega herramientas tradicionales como ifconfig y netstat, útiles para diagnóstico de red.
- curl: permite realizar pruebas de conectividad HTTP y verificar salida hacia servicios externos si es necesario.
- telnet: permite probar manualmente conexiones TCP hacia un puerto específico, por ejemplo el puerto 8080 del servidor.
- rm -rf /var/lib/apt/lists/*: elimina archivos temporales de apt para reducir el tamaño final de la imagen.
- WORKDIR /app: define el directorio de trabajo donde se colocan, compilan y ejecutan los archivos de la aplicación.

Host	IP	Gateway	Rol
ServerApp	192.168.1.130/26	192.168.1.129	Servidor TCP puerto 8080
ClientApp	192.168.1.190/26	192.168.1.129	Cliente TCP

```
# ServerApp
ip addr add 192.168.1.130/26 dev eth0
ip route add default via 192.168.1.129

# ClientApp
ip addr add 192.168.1.190/26 dev eth0
ip route add default via 192.168.1.129
```

En Docker, si el contenedor se recrea, la configuración manual de IP puede perderse. Por eso se registraron los comandos usados para poder repetir la configuración.

Figura 21. Comunicación entre los hosts de aplicación en la VLAN APP.

```
/app # ping 192.168.1.190
PING 192.168.1.190 (192.168.1.190): 56 data bytes
64 bytes from 192.168.1.190: seq=0 ttl=64 time=0.783 ms
64 bytes from 192.168.1.190: seq=1 ttl=64 time=0.725 ms
64 bytes from 192.168.1.190: seq=2 ttl=64 time=0.824 ms
64 bytes from 192.168.1.190: seq=3 ttl=64 time=0.883 ms
64 bytes from 192.168.1.190: seq=4 ttl=64 time=0.867 ms
64 bytes from 192.168.1.190: seq=5 ttl=64 time=0.736 ms
^C
--- 192.168.1.190 ping statistics ---
6 packets transmitted, 6 packets received, 0% packet loss
round-trip min/avg/max = 0.725/0.803/0.883 ms
```

9. Aplicación en C++ con sockets TCP

Se programó una aplicación cliente-servidor simple en C++. El servidor queda escuchando en el puerto 8080, recibe un mensaje del cliente y responde con un texto. La prueba confirma que el tráfico permitido por ACL funciona correctamente.

9.1 Código del servidor

```
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int server_fd, client_fd;
    sockaddr_in server_addr{}, client_addr{};
    socklen_t client_len = sizeof(client_addr);

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(8080);

    bind(server_fd, (sockaddr*)&server_addr, sizeof(server_addr));
    listen(server_fd, 1);

    std::cout << "Servidor escuchando en puerto 8080...\n";

    client_fd = accept(server_fd, (sockaddr*)&client_addr, &client_len);

    char buffer[1024] = {0};
    recv(client_fd, buffer, sizeof(buffer), 0);

    std::cout << "Mensaje recibido: " << buffer << std::endl;
```

```

    const char* respuesta = "Hola desde el servidor";
    send(client_fd, respuesta, strlen(respuesta), 0);

    close(client_fd);
    close(server_fd);

    return 0;
}

```

9.2 Código del cliente

```

#include <iostream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int sock;
    sockaddr_in server_addr{};

    sock = socket(AF_INET, SOCK_STREAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(8080);

    inet_pton(AF_INET, "192.168.1.130", &server_addr.sin_addr);

    connect(sock, (sockaddr*)&server_addr, sizeof(server_addr));

    const char* mensaje = "Hola servidor";
    send(sock, mensaje, strlen(mensaje), 0);

    char buffer[1024] = {0};
    recv(sock, buffer, sizeof(buffer), 0);

    std::cout << "Respuesta del servidor: " << buffer << std::endl;

    close(sock);

    return 0;
}

```

```

# Compilación
g++ serverApp.cpp -o servidor
g++ clienteApp.cpp -o cliente

# Ejecución
./servidor
./cliente

```

Figura 22. Servidor escuchando y recibiendo el mensaje.

```

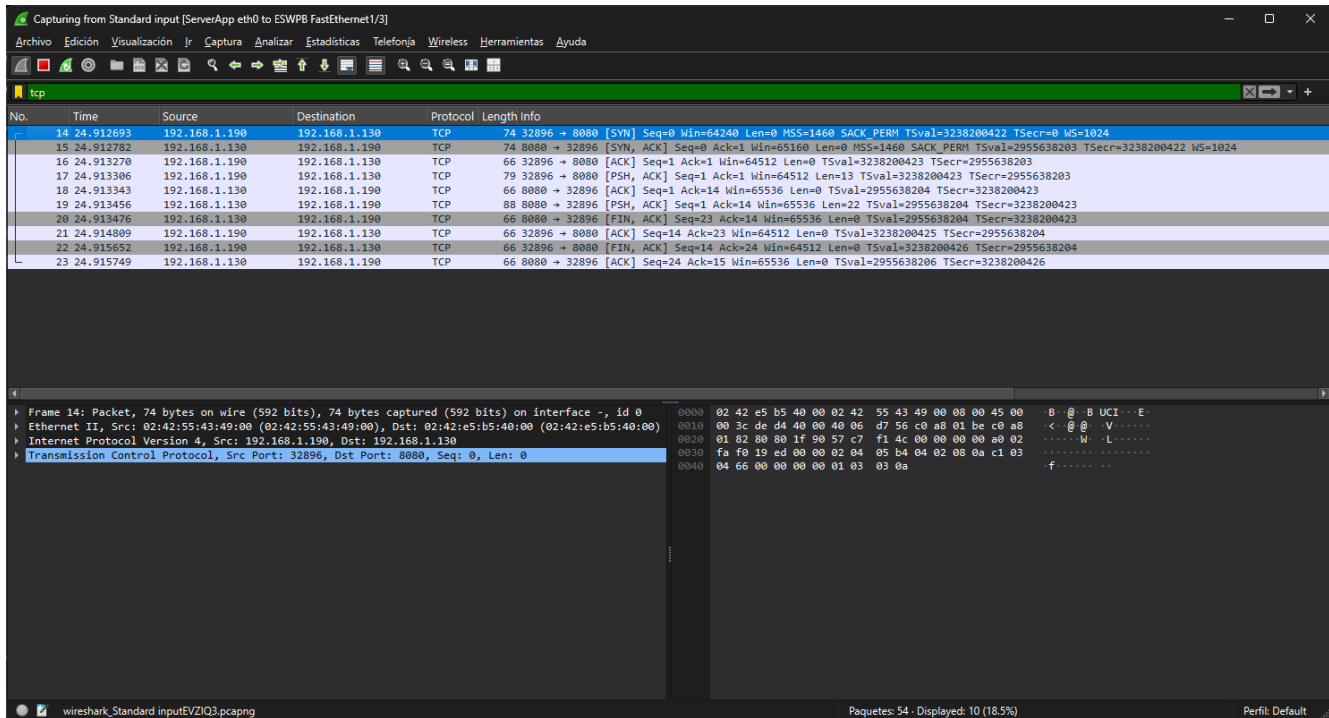
/app # ./server
Servidor escuchando en puerto 8080...
Mensaje recibido: Hola servidor

```


Figura 23. Cliente recibiendo respuesta

```
/app # ./cliente
Respuesta del servidor: Hola desde el servidor
```

Figura 24. Wireshark mostrando 3-way Handshake



10. Salida a Internet mediante NAT/PAT

Para dar salida a Internet se conectó FastEthernet1/0 del router RSJ al nodo NAT de GNS3. Esa interfaz recibió dirección por DHCP dentro de la red 192.168.122.0/24. El gateway entregado por el nodo NAT fue 192.168.122.1.

```
interface FastEthernet1/0
ip address dhcp
ip nat outside
ip virtual-reassembly
duplex auto
speed auto
```

La ruta por defecto se configuró apuntando al gateway del nodo NAT. Esta opción funcionó correctamente, a diferencia de apuntar sólo a la interfaz de salida.

```
ip route 0.0.0.0 0.0.0.0 192.168.122.1
```

El acceso a Internet se limitó a ServerApp y ClientApp mediante una ACL estándar usada por NAT.

```
ip access-list standard NAT_SERVERS
permit 192.168.1.190
permit 192.168.1.130

ip nat inside source list NAT_SERVERS interface FastEthernet1/0 overload
```

La palabra overload indica PAT. Con PAT, ambos equipos pueden compartir la misma IP de salida del router, diferenciándose por los puertos de origen. Esto cumple el requisito de NAT usando una única dirección de salida.

Pro	Inside global	Inside local	Outside local	Outside global
icmp	192.168.122.156:53	192.168.1.130:53	8.8.8.8:53	8.8.8.8:53

Figura 25. Configuración de NAT hacia la interfaz

```
RSJ#show run | section nat
ip nat inside
ip nat outside
ip nat inside source list NAT SERVERS interface FastEthernet1/0 overload
```

Figura 26. Traducciones NAT

```
RSJ#show ip nat translations
Pro Inside global      Inside local      Outside local      Outside global
icmp 192.168.122.156:72 192.168.1.130:72  8.8.8.8:72         8.8.8.8:72
icmp 192.168.122.156:353 192.168.1.190:353 8.8.8.8:353        8.8.8.8:353
```

Figura 27. Ping 8.8.8.8 desde serverApp

```
/app # ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=115 time=74.880 ms
64 bytes from 8.8.8.8: seq=1 ttl=115 time=57.685 ms
64 bytes from 8.8.8.8: seq=2 ttl=115 time=52.814 ms
64 bytes from 8.8.8.8: seq=3 ttl=115 time=63.586 ms
^C
--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 52.814/62.241/74.880 ms
```

Figura 28. Ping 8.8.8.8 desde clientApp

```
/app # ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=115 time=93.393 ms
64 bytes from 8.8.8.8: seq=1 ttl=115 time=47.034 ms
^C
--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 47.034/70.213/93.393 ms
```

11. DNS

En ServerApp y ClientApp se configuraron servidores DNS públicos para resolver nombres de dominio. En las pruebas se usaron 8.8.8.8 y 1.1.1.1.

```
echo "nameserver 8.8.8.8" > /etc/resolv.conf
echo "nameserver 1.1.1.1" >> /etc/resolv.conf
cat /etc/resolv.conf
```

11.1 Diferencia entre conectividad por IP pública y resolución DNS

Probar conectividad contra una IP pública, por ejemplo con ping 8.8.8.8, permite confirmar que existe salida a Internet a nivel de red. En esta prueba no interviene DNS, porque el destino ya está expresado directamente como una dirección IP. Si esta prueba funciona, quedan validadas la ruta por defecto, el gateway y NAT/PAT.

Probar con un nombre de dominio, por ejemplo ping google.com o nslookup google.com, valida un paso adicional: la resolución DNS. Antes de enviar paquetes al destino, el equipo debe consultar un servidor DNS para traducir el

nombre a una dirección IP. Por eso puede haber conectividad hacia 8.8.8.8 y, aun así, fallar google.com si DNS está mal configurado.

Figura 29. Ping google.com desde serverApp

```
/app # ping google.com
PING google.com (142.251.128.142): 56 data bytes
64 bytes from 142.251.128.142: seq=0 ttl=115 time=79.639 ms
64 bytes from 142.251.128.142: seq=1 ttl=115 time=43.784 ms
64 bytes from 142.251.128.142: seq=2 ttl=115 time=59.005 ms
64 bytes from 142.251.128.142: seq=3 ttl=115 time=43.779 ms
^C
--- google.com ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 43.779/56.551/79.639 ms
```

Figura 30. Ping google.com desde clientApp

```
/app # ping google.com
PING google.com (142.251.128.78): 56 data bytes
64 bytes from 142.251.128.78: seq=0 ttl=114 time=60.691 ms
64 bytes from 142.251.128.78: seq=1 ttl=114 time=45.423 ms
64 bytes from 142.251.128.78: seq=2 ttl=114 time=70.867 ms
64 bytes from 142.251.128.78: seq=3 ttl=114 time=54.336 ms
^C
--- google.com ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 45.423/57.829/70.867 ms
```

12. Problemas encontrados durante la implementación

12.1 DHCP bloqueado por ACL

Al principio los hosts de Ventas no obtenían IP por DHCP. El router tenía los pools creados y la VLAN respondía al gateway, pero la solicitud DHCP no llegaba al proceso DHCP del router. El problema era que la ACL aplicada a la subinterfaz filtraba el tráfico antes de que el cliente tuviera IP asignada. Se agregaron reglas para permitir UDP 67 y 68.

12.2 Interfaz trunk incorrecta

Se intentó configurar trunk en una interfaz que no correspondía al puerto de switch. Luego se identificaron los puertos correctos del módulo ESW y se verificó el trunk con show interfaces trunk.

12.3 Ruta por defecto hacia Internet

La ruta por defecto se configuró inicialmente apuntando a FastEthernet1/0. El router obtenía IP por DHCP y podía hacer ping al gateway 192.168.122.1, pero no llegaba a 8.8.8.8. Se corrigió configurando la ruta contra el next-hop 192.168.122.1.

12.4 Persistencia en Docker

Los contenedores Docker pueden perder configuración manual si se recrean. Para evitar problemas durante las pruebas, se dejaron registrados los comandos de IP, gateway, DNS, compilación y ejecución de la aplicación.